Interweaving Strings

AlgoExpert · Hard · Recursion

The problem

- Can `three` be formed by interleaving `one` and `two` keeping each one's order?

Intuition

- Each prefix pair of the two strings either explains the next character of three or not, so memoise by indices.

Brute force vs optimal

- Recursion choosing a or b at each step → O(2^(n+m))
- Memoise on (i, j) → O(n·m)
- Bottom-up table

Approach

- 1. Recursive i and j into one and two
- 2. Next character of three must come from one or the other
- 3. Memoise failed states
- 4. Succeed when all are used

Worked example

```
one = algo, two = frog

three = fralgogo
```

Is three an interleaving of one and two?

Worked example



dp[i][j]: first i of one + first j of two make the first i+j of three

Worked example



dp[4][4] = True

Java solution

```java
public class InterweavingStrings {
    public static boolean interweavingStrings(String one, String two, String three) {
        int n = one.length(), m = two.length();
        if (n + m != three.length()) return false;
        boolean[] dp = new boolean[m + 1];
        for (int i = 0; i <= n; i++)
            for (int j = 0; j <= m; j++) {
                if (i == 0 && j == 0) { dp[0] = true; continue; }
                boolean fromOne = i > 0 && dp[j] && one.charAt(i - 1) == three.charAt(i + j - 1);
                boolean fromTwo = j > 0 && dp[j - 1] && two.charAt(j - 1) == three.charAt(i + j - 1);
                dp[j] = fromOne || fromTwo;
            }
        return dp[m];
    }
}
```

Complexity

- Time: O(n*m)
- Space: O(m)

Edge cases & pitfalls

- Lengths must add up
- Both strings may match the same letter
- Empty strings

Interview tips & follow-ups

- Mention 1-D DP

Recap

- Each prefix pair of the two strings either explains the next character of three or not, so memoise by indices.
- Complexity: O(n*m)